

Coding for Artificial Intelligence

Module 1, Week 2: Computational Logic

Problem Set 2.1

(Bonus / Extension Lab)

Computational Logic and Multi-Signal Decision Systems

Lab Project: The Climatic Risk Intelligence Module

Course	Python for AI (Module 1, Week 2)
Author	Dr. Prateek, Assistant Professor, School of AI
Format	In-class bonus practical, 1 hour 30 minutes total, inclusive of write-up and submission
Total Marks	50 (+15 Bonus)

This is a bonus/extension activity. Read this document in full before you begin writing code.
While attempting the assignment, kindly turn off your internet for your own benefit.

Contents

1	Objective	2
2	The Problem: Autonomous Environmental Safety Tiering	2
3	Technical Requirements	2
3.1	Telemetry Ingestion with Validation	2
3.2	The Heuristic Logic (The “Smart” Engine)	3
3.3	Boolean Truthiness and Short-Circuiting	3
4	Constraints (“Standards”)	3
5	Edge Cases to Handle	4
6	Challenge Task (Bonus, +15 Marks)	4
7	Suggested Time Allocation	5
8	Deliverables	5
	Academic Integrity	5

1 Objective

Problem Set 2 introduced conditional branching using single, independent signals (temperature alone). This bonus practical raises the level of difficulty by asking you to build a **multi-signal heuristic system**: a program that combines several sensor readings into one derived quantity and reasons about that quantity using more advanced language features.

By the end of this practical you should be able to:

- (i) use the **assignment expression** (the “walrus” operator, `:=`) to capture and validate input in a single step,
- (ii) rely on Python’s **truthiness** rules and **short-circuit evaluation** instead of explicit equality checks, and
- (iii) design a multi-branch decision system in which the branches are derived from a combination of inputs, not a single value.

Time budget. This entire activity — coding, testing, and writing up your submission — is designed to fit in **90 minutes**. A suggested breakdown is given in Section 7.

2 The Problem: Autonomous Environmental Safety Tiering

In an AI-driven smart city, sensors provide raw telemetry from a human–robot collaborative work-site. Your task is to write the decision logic for a safety module that classifies the site into one of four tiers — **Operational**, **Cautionary**, **Critical**, or **Freeze Alert** — based on a derived **Heat Stress Index (HSI)**.

3 Technical Requirements

Your script, `assignment2.1_climatic_logic.py`, must implement the following.

3.1 Telemetry Ingestion with Validation

1. Prompt for **temperature** (degrees Celsius) and **humidity** (percentage).
2. **Constraint:** use the walrus operator inside an `if` statement to capture the raw input and immediately check that it is non-empty, in a single expression:

```
1 if (temp_str := input("Temperature (C): ").strip()):
2     ...
3 else:
4     print("Safety Level: Unknown")
```

3. **Numeric conversion — read this carefully.** Once you know the string is non-empty, you still need to convert it to a number. `str.isdigit()`, used in earlier problem sets, **cannot** correctly validate this input, because it returns `False` for negative numbers such as `"-5"` — and sub-zero temperatures are exactly what triggers your Freeze Alert tier. Use a `try/except ValueError` block around your `float()` conversion instead, and treat a conversion failure the same way as empty input: assign `"Safety Level: Unknown"` and terminate gracefully. The walrus operator handles the *emptiness* check; the `try/except` block handles *validity* of a non-empty string. These are two distinct failure modes and both must be handled.

3.2 The Heuristic Logic (The “Smart” Engine)

Calculate the Heat Stress Index using this simplified model:

$$HSI = \text{Temperature} + (0.5 \times \text{Humidity}) \quad (1)$$

Categorize the environment using a single `if/elif/else` chain, evaluated **in the exact order given below**. Order matters here: a work-site at 0°C with very high humidity produces a numerically large HSI, but it is still a freezing environment, not a heat emergency. Checking Freeze Alert first prevents this kind of misclassification.

Tier (checked in order)	Condition
1. FREEZE ALERT	Temperature ≤ 0
2. CRITICAL	$HSI > 45$, or (Temperature > 38 and Humidity > 70)
3. CAUTIONARY	$30 \leq HSI \leq 45$, and wind speed (a third input, km/h) < 5
4. OPERATIONAL	everything else

Note on the Operational tier. Rather than writing a fourth explicit condition (e.g. “HSI above 15”), let **Operational** be a plain **else** — the catch-all for any reading that is not a freeze, a critical heat emergency, or a wind-limited cautionary case. This is both more elegant (fewer comparisons, satisfying the Logic Efficiency requirement in Section 4) and more correct: an explicit lower-bound condition would leave certain low-HSI, non-freezing readings unclassified by accident.

Worked Examples. These four cases should help you verify your implementation.

Temp	Humidity	Wind	HSI	Expected Tier
−5	50	—	20.0	FREEZE ALERT (checked first, regardless of HSI)
32	80	—	72.0	CRITICAL ($HSI > 45$)
25	20	3	35.0	CAUTIONARY ($30 \leq 35 \leq 45$, wind < 5)
20	10	10	25.0	OPERATIONAL (fails all prior conditions)

3.3 Boolean Truthiness and Short-Circuiting

1. Rather than testing `if input_string == ""`, use Python’s inherent truth values: an empty string is “falsy”, so `if not user_input:` is the idiomatic way to detect it.
2. Use the `or` operator to provide a default value when a field is left blank:

```
1 user_name = input("Name: ") or "Guest_User"
```

This works because `or` returns its *first truthy operand*: if `input()` returns an empty string (falsy), Python evaluates and returns `"Guest_User"` instead.

4 Constraints (“Standards”)

To satisfy the requirements of a top-tier technical institution, your solution will be graded on:

1. **Logic Efficiency.** Minimise the number of comparisons. Do not re-check a condition already ruled out by an earlier branch in the same chain (e.g., inside the **CAUTIONARY** branch you already know HSI is not > 45 , so do not test that again).
2. **Syntactic Elegance.** Use **ternary expressions** for simple binary choices, e.g.:

```
1 risk_label = "Safe" if hsi < 30 else "Unsafe"
```

3. **PEP 8 Compliance.** All variables in `snake_case`; exactly one space around comparison operators (`hsi >= 45`, never `hsi>=45`).
4. **Docstring Quality.** A professional header docstring explaining the Heuristic Tiering Model, the order-of-evaluation rationale, and your author information.

5 Edge Cases to Handle

1. **The “Zero-Value” Case.** What happens if the user enters 0 for temperature? Note that the integer 0 is “falsy” in Python, so a careless `if temperature:` check would treat a genuine, valid 0°C reading as though it were missing input. Your emptiness check must operate on the *raw input string* (before conversion), never on the converted number, precisely to avoid this trap.
2. **Input Robustness.** What happens if humidity is entered as a negative number? Add a sanity-check assertion:

```
1 assert humidity >= 0, "Telemetry Error: Negative Humidity"
```

Note that an `assert` failure raises an `AssertionError` and stops the program; this is appropriate here because negative humidity is not a valid physical reading, and is different in kind from a user simply leaving a field blank.

6 Challenge Task (Bonus, +15 Marks)

Implement a **Nested Intelligence Suite**. If, and only if, the environment is classified as **CAUTIONARY**, prompt for a `battery_level` (percentage) and adjust the tier as follows:

1. If `battery_level < 20%`: elevate the status to **CRITICAL** (risk of robot failure in sustained heat).
2. If `battery_level > 80%`: downgrade the status to **OPERATIONAL** (robots can sustain high-load cooling).
3. Otherwise: leave the status at **CAUTIONARY**.

Hint. This adjustment should happen *after* the primary tier has already been determined, as a nested check inside the **CAUTIONARY** branch only — do not prompt for battery level if the site is already **FREEZE ALERT**, **CRITICAL**, or **OPERATIONAL**.

7 Suggested Time Allocation

Phase	Description	Duration
Core Logic	Telemetry validation, HSI calculation, tiering chain	45 minutes
Testing	Verify against the four worked examples; try your own edge cases	25 minutes
Documentation & Submission	Docstring, execution trace, final write-up	20 minutes
Total		90 minutes

8 Deliverables

1. **Source Code:** A single file named `assignment2_1_climatic_logic.py`.
2. **Execution Trace:** A plain-text file showing the logic path for three scenarios:
 - (a) an empty input correctly handled by the walrus-operator check,
 - (b) a **Critical** heatwave reading, and
 - (c) a **Cautionary** reading that is then adjusted by the Battery-Level bonus logic.
3. **Submission Documentation:** A short note (150–200 words), included either as an extended docstring or a separate paragraph, explaining *why* the tiers are checked in the specific order given in Section 3.2, and how your empty-input check avoids the “Zero-Value” trap described in Section 4.

Academic Integrity

As a bonus activity, this practical may be discussed with peers at a conceptual level, but the code and documentation you submit must be written independently and must reflect your own understanding. Submissions found to be substantially copied will be treated in accordance with the institute’s academic integrity policy.